Finite Descent Authority: Packets, Pipelines, and Seven Halt Guards (CODA-FDA)

Subtitle: A proposal is not a decision

Author: Jeremy H. Carroll

Date: 2026-09-28

Version: v0.1.1 Preprint

License: Creative Commons Attribution 4.0 International

CODA Architecture & Layers

CODA stands for the Calculus of Obstruction, Descent, and Authorization. It is an exact, finite mathematical apparatus that enforces strict claim discipline by separating linear repairs, transport certificates, and finite descent from ungrounded authority claims. The system comprises eight operational layers, FOC through A, under the GTLA meta-architecture.

Layer	Abbreviation	Layer / Component Name & Scope
1	**FOC**	Finite Obstruction Calculus
2	**MUR**	Meta & Universal Re-Presentation (what presentations exist)
3	**CS**	Continuum Shadow (what continuum presentation may descend to a finite calculus system)
4	**CS_Q**	Continuum Shadow to Continuum Bridge (how continuum presentations descend)
5	**Q**	The Continuum (continuum presentations via completion and enclosure)
6	**A**	Anomalon (global residuals not removed and typed instead)
7	**OR**	Omniversal Re-Presentation (what presentations can transport)
8	**FDA**	Finite Descent Authority (what presentation may descend to evaluation and authorization)
9	**GTLA**	Layered Authority Architecture for CODA

Abstract

A proposal is not a decision. Standing, in this paper, is a recorded pair: the pipeline actually produced a terminal t, and seven halt guards derived from evidence all hold. The data that can support that pair is a five-field packet (K, E, V, τ, Λ). Without a packet there is no standing. A confidence score, a fluent paragraph, or a successful route does not inhabit the pair. One of the seven guards may read leftover mass from an exact rational cochain solver; that solver measures leftover, and is not a theorem about where standing comes from. If the pipeline returns a terminal, it has a history. If routing already refused, later stages cannot overwrite that refusal. In the regime model, soundness

of standing ($\mathsf{Standing}_R$) for truth is an extra pair of hypotheses, and there is a countermodel when those hypotheses fail. The operational pipeline record of Definition FDA-D2 is not connected to truth without a declared sound regime embedding. The Python companion includes a policy evaluator over caller attestations and a separate finite execution harness. The harness records every candidate, checks certificates against a declared repair map, compares serialized skeleton snapshots, reads a declared lift registry, and compares two execution traces. These checks do not establish semantic verifier soundness. "Standing" is an internal witness type in the declared formal setting. This work confers no legal, institutional, safety, publication, or real-world authority;

Introduction

It is easy to treat a fluent answer as a decision. A model writes a function, attaches a confidence of 0.99, and a downstream system ships it. Nothing in that number says that a verifier ran, that the same evidence replays, that every declared candidate was checked, or that leftover mass is zero. The number is a score. Standing, as defined here, is not a score.

A picture. There is a proposal p: an answer, a patch, a classification, analogous to proof-carrying code [2]. A *packet* is five pieces of finite data that say what world you are in, which candidates are allowed, how they are checked, how a check becomes a terminal, and what happens on failure. A *pipeline* is four maps in order — route, descend, verify, terminalize — each of which may refuse. Standing is inhabited only when the pipeline returns a left injection $\mathsf{inl}(t)$ *and* seven derived halt guards hold of t. A supplied packet is represented by an optional input. If that input is `none`, the corresponding supplied-standing type is empty (FDA-T2).

The theory is refuted by any of: an inhabitant of standing when no packet exists; an inhabitant on a refusal branch; an inhabitant whose seven-guard conjunction fails; a successful terminal with no intermediate history; a routing error overwritten by a later success; a claim that operational standing implies truth without a sound regime embedding and the two extra hypotheses of Theorem FDA-T7; or a claim that the cochain leftover solver is an origin-of-standing theorem.

Notation

- p is a proposal. It is an input, not a decision.
- K is a finite carrier: the context in which candidates live.
- E is a family of admissible candidates over K.
- V is a local verifier: given a carrier point and a candidate, it produces an evidence term, or fails.
- τ is a terminalizer: it turns evidence into a terminal outcome.
- Λ is the failure discipline: continuation rules (refusal, lift, or rollback on the same carrier) specifying how non-promotion is handled, not a silent promotion. Verification failure itself is eliminated into a refusal branch by the pipeline error monad.
- $\mathbb{T}$ is the type of terminals; Ref is the type of refusals. The pipeline returns a sum $\mathbb{T} + \mathrm{Ref}$.
- $\mathsf{inl}(t)$ is the left injection: a terminal-branch value. $\mathsf{inr}(r)$ is a refusal.
- $g_1, \ldots, g_7$ are seven halt guards, derived from a finite evidence record, not supplied as unbound booleans.
- $\mathsf{IsAuth}(t)$ is their conjunction.
- $\Phi_1([x])$ is the cheapest leftover of a defect x after licensed linear repairs, as in the companion finite obstruction calculus [3].

A **five-field packet** is the tuple (K, E, V, τ, Λ).

1. Five-field packets

Definition FDA-D1 — packet

A packet consists of

$$K, \qquad E\colon K \to \mathrm{Type}, \qquad V\colon (k : K) \to E(k) \to \mathrm{Type},$$

$$\tau\colon \{k : K\} \to \{e : E(k)\} \to V(k, e) \to \mathbb{T}, \qquad \Lambda\colon \{k : K\} \to (e : E(k)) \to V(k, e) \to \mathrm{Type}.$$

In words: K says where you are; E says which candidates may be tried there; V is the check; τ is the exit into a terminal; Λ is what continuation rules are indexed by successful verifier witnesses. The Lean record is `DescentPacket`. The Python record is `DescentPacket` with string and immutable candidate-ID tuple fields. Runtime validity requires nonblank carrier/verifier/terminalizer/discipline names and a nonempty tuple of distinct, nonblank candidate IDs. The Lean record itself allows empty types and does not encode these runtime validity or finiteness conditions.

The Python companion refuses invalid fields or a missing packet before building a successful result. This runtime validation is separate from the Lean packet type, which has an inhabitant with carrier `Empty` for every terminal type.

The five fields are the standing-facing data. They are not the eight-field trace packet of the obstruction calculus. A projection from a richer trace record to this five-field packet is a separate witness, not an identification of field names.

2. Standing is a dependent pair

A sealed pipeline, given a packet and a proposal, returns either a terminal or a refusal:

$$\Pi^\sharp\colon \mathrm{Packet} \to \mathrm{Proposal} \to \mathbb{T} + \mathrm{Ref}\,.$$

The reconstruction companion writes the same idea as four staged maps, each valued in an error monad:

$$\mathrm{route}\colon P \to S + F, \quad \mathrm{descend}\colon S \to P \to X + F,$$

$$\mathrm{verify}\colon S \to X \to E + F, \quad \mathrm{terminalize}\colon S \to X \to E \to \mathbb{T} + F.$$

The composite `run` tries them in that order and stops at the first refusal.

Definition FDA-D2 — standing

Standing of a proposal p against a packet and a pipeline is the dependent pair (in the sense of Martin-Löf [1]):

$$\mathsf{Standing}(p) := \Sigma(t : \mathbb{T})(\Pi^\sharp(\mathrm{packet}, p) = \mathsf{inl}(t) \ \wedge \ \mathsf{IsAuth}(t)).$$

In words: there is a terminal t, the pipeline actually produced that t (not a refusal, not a guessed label), and the seven halt guards hold of t. The Lean record is named `Authority`. It has three fields: `terminal`,

a path equality `terminalizerPath`, and `isAuth`. The packet and the proposal are indices of the record, not optional attachments.

A generated candidate may carry a claimed terminal and even a valid guard witness. It is a different type. It has no pipeline path, so it is not standing.

3. Seven guards derived from evidence

Definition FDA-D3 — derived guards

Let H be a finite evidence record with the following fields: whether the pipeline accepted the run; whether replay was deterministic; a non-negative rational residual mass; whether the declared skeleton projection has the same canonical value at each stage boundary; which candidate identifiers were verified against declared candidates; how many lifts are pending; whether a refusal is active. The seven guards are *derived*:

1. g_1 `accepted` — the pipeline accepted the run.
2. g_2 `deterministic` — replay of the same evidence is identical.
3. g_3 `zero_residual_mass` — residual mass is exactly 0.
4. g_4 `skeleton_preserving` — canonical snapshots of the declared skeleton projection agree at all measured stage boundaries. The default projection is the whole routed state. This does not observe transient changes between checks.
5. g_5 `exhaustive` — all declared candidate identifiers in E are verified, and at least one candidate was declared. Checking zero of zero is not exhaustive.
6. g_6 `no_pending_lift` — the pending-lift count is 0.
7. g_7 `no_refusal` — no active refusal sits on the result.

Then

$$\mathsf{IsAuth}(t) : \Longleftrightarrow\ g_1(t) \wedge g_2(t) \wedge g_3(t) \wedge g_4(t) \wedge g_5(t) \wedge g_6(t) \wedge g_7(t).$$

The preferred companion entry `evaluate_authorization` takes evidence and calls `derive_guards`. It does not accept IsAuth as a single boolean. A compatibility helper accepts a pre-built seven-tuple for tests; that helper is not a licence to skip derivation.

Theorem FDA-T1 — one failed guard kills standing

If $\neg\mathsf{IsAuth}(\mathrm{guards}(t))$, then there is no standing record whose terminal equals t.

Proof. A standing record supplies `isAuth : IsAuth (guards terminal)`. Transport along the assumed equality of terminals. Contradiction. This is Lean `failed_guard_non_creation`. □

In the companion, flipping any one evidence field so that its derived guard fails makes `inhabited` false. The tests do this for each of the seven.

4. No packet, no standing

Theorem FDA-T2 — no supplied packet, no standing

Let the supplied input have type `Option (DescentPacket Terminal)`. `SuppliedAuthority input p` contains a packet, an equality `input = some packet`, and `Authority` for that packet and proposal. Then `SuppliedAuthority none p` is empty.

Proof. An inhabitant supplies `none = some packet`, contradicting the disjoint constructors of `Option`. This is Lean `missing_supplied_packet_non_creation`. □

The retained historical theorem `uniform_non_creation` assumes that the entire packet type is empty. That hypothesis is impossible here: choose carrier `Empty` and define every dependent field by empty elimination. It is not the formal model of an absent supplied input. Runtime nonblank names and candidate validation are additional Python conditions, not consequences of FDA-T2.

Theorem FDA-T3 — no terminal path, no standing

If every terminal t satisfies $\Pi^\sharp(\text{packet}, p) \neq \mathsf{inl}(t)$, then standing is empty for that packet and proposal.

Proof. An inhabitant supplies a path equality to $\mathsf{inl}(t)$. This is Lean `authority_non_construction`. □

Corollary FDA-C1 — a refusal is not standing

If $\Pi^\sharp(\text{packet}, p) = \mathsf{inr}(r)$, then standing is empty.

Proof. The sums inl and inr are disjoint. Apply FDA-T3. This is Lean `refusal_non_creation`. □

Theorem FDA-T4 — a generated candidate is not standing

Let c be a generated candidate with terminal t_c and a valid guard witness. If the pipeline does not return $\mathsf{inl}(t_c)$, then no standing record has terminal t_c.

Proof. A standing record with that terminal would give a path equality, contradicting the missing path. This is Lean `authority_non_generation`. □

5. Fluency and confidence are not standing

The standing type of Definition FDA-D2 has three ingredients: a terminal, a pipeline path, and IsAuth. It has no field for a logit, a benchmark score, a majority vote, a routing success flag, or the fluency of a paragraph. A number in $[0, 1]$ cannot inhabit a dependent pair that does not mention it.

In the companion, `evaluate_authorization` accepts an optional `logit_confidence` and discards it. With `packet = None` and confidence 999/1000, the result is `inhabited = False` and refusal code `R_nopacket`. The same refusal occurs if the packet is present but invalid (empty carrier). Green guards do not help: the missing-packet branch is taken first.

That is the whole claim. Fluency can be useful as a search heuristic. It is not a path equality, and it is not seven derived guards.

6. Residual mass from a leftover calculator

Guard g_3 needs a non-negative rational residual mass. One lawful source of that number is the leftover of a finite linear cochain problem solved via exact linear programming duality (Schrijver [4], Boyd and Vandenberghe [5]), as constructed in the companion finite obstruction calculus (CODA-FOC [3]). Let d_0 be a repair matrix with rational entries, x a defect vector, and w a strictly positive rational weight vector. Set

$$\Phi_1([x]) = \min_f \|x + d_0 f\|_{1,w}.$$

When x lies in the image of d_0, the leftover is 0 and g_3 can hold. When it does not, the leftover is positive and g_3 fails. The companion function `compute_cochain_residual_quotient` calls `LinearRepairProblem` and returns an exact rational primal–dual certificate for the unweighted norm ($w_i = 1$); the implementation has no general weight-vector argument. A two-row example:

- $d_0 = (1, -1)^T$, $x = (2, -2)^T = 2 \cdot d_0$: leftover 0.
- $d_0 = (1, -1)^T$, $x = (1, 1)^T$: leftover 2 in unweighted ℓ^1.

Proposition FDA-P1 — the solver is not an origin theorem

Attaching $\Phi_1([x])$ to the residual-mass field of the evidence record feeds guard g_3. It does not construct a cochain of standing, does not prove that standing originates in homology, and does not replace the path equality of Definition FDA-D2. The leftover calculator may be correct as a leftover calculator and still not be a theorem about the origin of standing.

The companion also stamps small diagnostic rationals on refusal branches (1 for no packet, 1/2 for a failed g_3, 1/5 for other guard failures, 0 for inhabited standing). Those stamps are labels in the Python result object. They are not Φ_1.

7. Reconstruction: history, refusal, soundness, countermodel

The reconstruction file supplies a four-stage pipeline and a small regime record. A regime R has four predicates: `verifies`, `recordedGrant`, `truth`, and `entitled`. Standing in that language is

$$\mathsf{Standing}_R(g, p) : \Longleftrightarrow \ (\exists e.\ R.\mathrm{verifies}(p, e)) \ \wedge \ R.\mathrm{recordedGrant}(g, p).$$

Observation (a factor through a collapsing map) is a different type. It is not this conjunction.

Theorem FDA-T5 — a terminal has a history

If $\mathrm{run}(p) = \mathsf{ok}(t)$, then there exist a sector s, descended data x, and evidence e such that

$$\mathrm{route}(p) = \mathsf{ok}(s), \quad \mathrm{descend}(s, p) = \mathsf{ok}(x),$$

$$\mathrm{verify}(s, x) = \mathsf{ok}(e), \quad \mathrm{terminalize}(s, x, e) = \mathsf{ok}(t).$$

Proof. Unfold `run`. Each of the first three stages is a match on `ok` / `error`. An `error` branch makes `run` return `error`, contradicting `ok(t)`. The remaining branch is the witness. This is Lean `terminal_has_history`. □

Theorem FDA-T6 — routing refusal persists

If $\mathrm{route}(p) = \mathsf{error}(f)$, then $\mathrm{run}(p) = \mathsf{error}(f)$.

Proof. Unfold `run`. The first match returns `error(f)`. Later stages are not called. This is Lean `routing_refusal_persists`. □

Theorem FDA-T7 — standing is sound under two extra hypotheses

Assume

$$\forall p, e.\ R.\text{verifies}(p, e) \Rightarrow R.\text{truth}(p),$$

$$\forall g, p.\ R.\text{recordedGrant}(g, p) \Rightarrow R.\text{entitled}(g, p).$$

If $\mathsf{Standing}_R(g, p)$, then $R.\text{truth}(p)$ and $R.\text{entitled}(g, p)$.

Proof. Standing supplies some e with `verifies` and a recorded grant. Apply the two hypotheses. This is Lean `standing_sound`. □

The two hypotheses are not supplied by inhabiting the pair. They are extra. The theorem is an implication, not a verifier-soundness proof.

Connection between Operational Standing (FDA-D2) and Regime Standing ($\mathsf{Standing}_R$)

Definition FDA-D2 defines standing $\mathsf{Standing}(p)$ as an internal operational pipeline record: a dependent pair of a terminal $t \in \mathbb{T}$, a pipeline path equality $\Pi^\sharp(\text{packet}, p) = \mathsf{inl}(t)$, and the conjunction $\mathsf{IsAuth}(t)$ of seven derived halt guards. In contrast, $\mathsf{Standing}_R(g, p)$ in this section models an epistemic and entitlement regime over abstract verification, grant, truth, and entitlement predicates.

Theorem FDA-T7 proves soundness strictly for the regime predicate $\mathsf{Standing}_R(g, p)$ under explicit verifier-soundness and grant-soundness hypotheses within R. The supplied Lean formalization does not connect the operational FDA-D2 record to $\mathsf{Standing}_R$. This distinction is mathematically essential: a constant-success operational pipeline with all seven halt guards trivially true can inhabit FDA-D2 while an unrelated regime R has everywhere-false verification, grant, and truth predicates. In that disjoint scenario, both soundness hypotheses of Theorem FDA-T7 hold vacuously for R, yet this vacuous regime soundness confers no truth or entitlement on the operational FDA-D2 inhabitant.

To connect operational standing to regime soundness, a system must declare an explicit regime embedding R_Π. The canonical embedding sets:

$$R_\Pi.\text{verifies}(p, e) :\iff \exists s, x.\ \text{route}(p) = \mathsf{ok}(s)\ \wedge\ \text{descend}(s, p) = \mathsf{ok}(x)\ \wedge\ \text{verify}(s, x) = \mathsf{ok}(e),$$

$$R_\Pi.\text{recordedGrant}(t, p) :\iff \Pi^\sharp(\text{packet}, p) = \mathsf{inl}(t)\ \wedge\ \mathsf{IsAuth}(t).$$

By Theorem FDA-T5 (`terminal_has_history`), an operational execution that reaches $\mathsf{inl}(t)$ with all guards satisfied supplies the intermediate evidence e that inhabits $\mathsf{Standing}_{R_\Pi}(t, p)$. Under this canonical embedding, soundness of the resulting terminal t for p holds if and only if the stage verifier is semantically sound ($\text{verify}(s, x) = \mathsf{ok}(e) \implies R_\Pi.\text{truth}(p)$) and the terminal authorization grant is sound ($\Pi^\sharp = \mathsf{inl}(t) \wedge \mathsf{IsAuth}(t) \implies R_\Pi.\text{entitled}(t, p)$). Absent such a declared and validated regime embedding, FDA-D2 operational standing certifies deterministic host completion and guard satisfaction only; it does not establish semantic truth, verifier soundness, or institutional authority.

Theorem FDA-T8 — untrusted verifier countermodel

There exists a regime R on the unit type such that $\mathsf{Standing}_R$ holds and $R.\text{truth}$ fails.

Proof. Take `verifies` constantly true, `recordedGrant` constantly true, `truth` constantly false, and `entitled` constantly true. Standing holds by the unit evidence and the unit grant. Truth is false. This is Lean `untrusted_verifier_countermodel`. □

So: standing without a sound verifier is not truth. The countermodel is deliberately trivial. That is the point. Nothing in the pair requires the verifier to track truth.

A bicategorical packaging of the same stages — a displayed bicategory of finite authorization systems, Kleisli pasting of refusals — has been sketched in companion notes. No bicategory axioms, pasting theorem, or Kleisli soundness result is proved here.

7.5 Authority Firewall Synthesis & Verification Limits

This section synthesizes the foundational results established across the 007_AUTHORITY_FIREWALL monograph series (Papers 004, 005, 027, 030):

> **Status of FDA-T9 to FDA-T12 (retyped 2026-09-23).** The ledger cited four Lean declarations for these items: `GTFDA.coproduct_separation`, `GTFDA.role_non_impersonation`, `GTFDA.authority_gap_fence` and `GTFDA.authority_typing_boundary`. None of them exists in `lean4/GtfdaAuthorityCore.lean`, and the Lean claim map covers FDA-D1 to FDA-T8 only. Those pointers are **WITHDRAWN 2026-09-23**. Each item below states what does back it. The headings keep their numbers, so citations to them still resolve.

Policy FDA-T9 — residual outcomes require an explicit promotion rule

Status. The refusal-branch result FDA-C1 is proved. The earlier categorical claim that constructor disjointness implies an empty set of natural transformations is withdrawn as false.

The terminal convention separates an authority constructor from a residual constructor. The package declares no rule that converts a residual outcome into standing; constructing standing still requires the path and guards of FDA-D2. This is a restriction of the declared interface.

Constructor disjointness does not prohibit maps between summands. For a concrete countermodel, take the category with one object and only its identity and two constant singleton functors. There is one natural transformation between those functors, although the two singleton injections into their coproduct remain distinct. FDA-C1 therefore supports refusal-branch exclusion, not a general categorical no-map theorem.

Policy FDA-T10 — Role Non-Impersonation & Rubber-Stamp Prohibition (Paper 004)

Status. A declared role contract, not a theorem. The nearest Lean result is FDA-T4 (`GTFDA.authority_non_generation`): a generated candidate without a terminalizer path is not standing.

Proposals and evidence evaluations do not constitute authority:

$$\boxed{p \in \mathcal{P} \not\Rightarrow \mathsf{Auth}(p)}$$

Under the declared Omega role contracts, the six tower positions $(\partial, \mathbb{A}, \chi, \mathfrak{O}, \kappa, \Lambda\Omega)$ form non-identifiable roles. An internal proposer generating candidates $p \in \mathcal{P}$, an evaluator computing Φ_1, or a transport functor computing $\tau^\sharp$ cannot impersonate an authority terminal. Any pipeline configuration where a proposal process serves as terminal authority for its own output is strictly prohibited (rubber-stamp prohibition).

Scope fence FDA-T11 — The Authority Gap & Exactness Fraud Criterion (Paper 005)

Status. A non-implication plus a routing policy. This paper defines no semantic-correctness predicate, so the non-implication is a scope fence, not a theorem. The routing sentence is a policy.

A vanishing quotient defect does not imply semantic correctness:

$$\Phi_1(\Omega) = 0 \not\Rightarrow \text{Semantic Correctness}$$

A degenerate observation can conceal distinctions that the selected quotient does not measure. No observer-erasure theorem is proved in this package. External institutional licensing is a separate application policy; the FDA runtime does not model or confer such a license.

Scope fence FDA-T12 — Verification Limits & Authority Typing Boundary (Paper 030)

Status. A scope fence. The formal content nearest to it is FDA-T7 (standing is sound only under explicit verifier-soundness and grant-soundness hypotheses) and FDA-T8 (a countermodel without them).

The package does not formalize verifier adequacy, an external authority root, or a general impossibility theorem for self-auditing. Its precise result is FDA-T7's conditional implication and FDA-T8's countermodel without soundness hypotheses. The labels `ContractAuthority`, `VerifierScopedAuthority`, and `SemanticAuthority` are policy vocabulary, not additional Lean witness types.

8. Computational companion

The production module `code/coda_fda_core.py` validates five-field packets, derives the seven guards from evidence, refuses a missing packet even at high confidence, and optionally attaches an exact rational leftover certificate to guard g_3. Arithmetic on residual mass is exact rational; floats and booleans in scalar position raise `TypeError`.

The separate `code/coda_fda_pipeline.py` executes the four stages. Its constructor requires an exact repair matrix and a callback exposing the finite pending-lift registry. A skeleton projection defaults to the whole routed state. Each candidate's descent, witness, and independently checked certificate are recorded; the residual policy is the maximum distance over every declared candidate. A positive distance blocks terminalization. The terminal uses the first candidate and its own certificate. Missing measurements, malformed certificates, state changes at measured boundaries, active lifts, and skipped replay block operational authorization.

The canonical trace serializes the packet, proposal, repair map, all stage events, all candidates, certificates, measured skeletons and lift IDs. Its SHA-256 digest is recomputable from that payload using sorted, compact, UTF-8 JSON. Container types are tagged, so user dictionaries cannot impersonate numeric encodings. The selected evidence is rechecked against its stored snapshot after later candidates run. A hash checks integrity of recorded data, not authenticity or truth. Replay compares two complete execution payloads under the same declared callbacks. Callback definitions, skeleton projection, lift-registry completeness, and external state are explicit model assumptions.

Lean files in `lean4/`:

- `GtfdaAuthorityCore.lean` — bounded standing core: packet, guards, standing record, and definitional emptiness theorems FDA-T1–T4;
- `FDA.lean` and `Algebra.lean` — wraps reconstruction for FDA-T5–T8;
- `Core.lean` — re-exports the files above.

There are no `sorry` stubs in those files. The reconstruction theorems use the ordinary Lean kernel. They do not prove that any particular verifier is sound, and they do not prove an origin-of-standing cochain.

Adversarial tests include: empty carrier; missing packet with confidence 999/1000; each single-guard failure; zero declared candidates; float rejection; strict boolean type enforcement; candidate identifier set binding; forged certificate rejection; leftover 0 and leftover 2 on the two-row cochain example.

9. Limitations

The packet carrier and candidate types in Lean are unrestricted `Type u` fields; the Lean mechanization establishes definitional structure laws, not finiteness bounds. The leftover solver is linear and finite-dimensional. Lean proves definitional emptiness and the reconstruction implications above. It does not prove verifier soundness, semantic packet soundness, or an origin-of-standing cochain. The leftover calculator of Section 6 is a leftover calculator. The policy evaluator consumes caller attestations. The separate execution harness checks finite records and declared callbacks at stage boundaries. Neither establishes semantic verifier soundness, absence of undeclared external work, or determinism beyond the two compared runs.

References

1. P. Martin-Löf, *Intuitionistic Type Theory*, Bibliopolis, 1984.
2. G. C. Necula, "Proof-carrying code," in *POPL*, 1997.
3. J. H. Carroll, *The Finite Obstruction Calculus: Exact quotient diagnostics, dual witnesses, two-stage coherence, and transport (CODA-FOC)*, 2026. https://doi.org/10.5281/zenodo.20195157
4. A. Schrijver, *Theory of Linear and Integer Programming*, Wiley, 1986.
5. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. https://web.stanford.edu/~boyd/cvxbook/